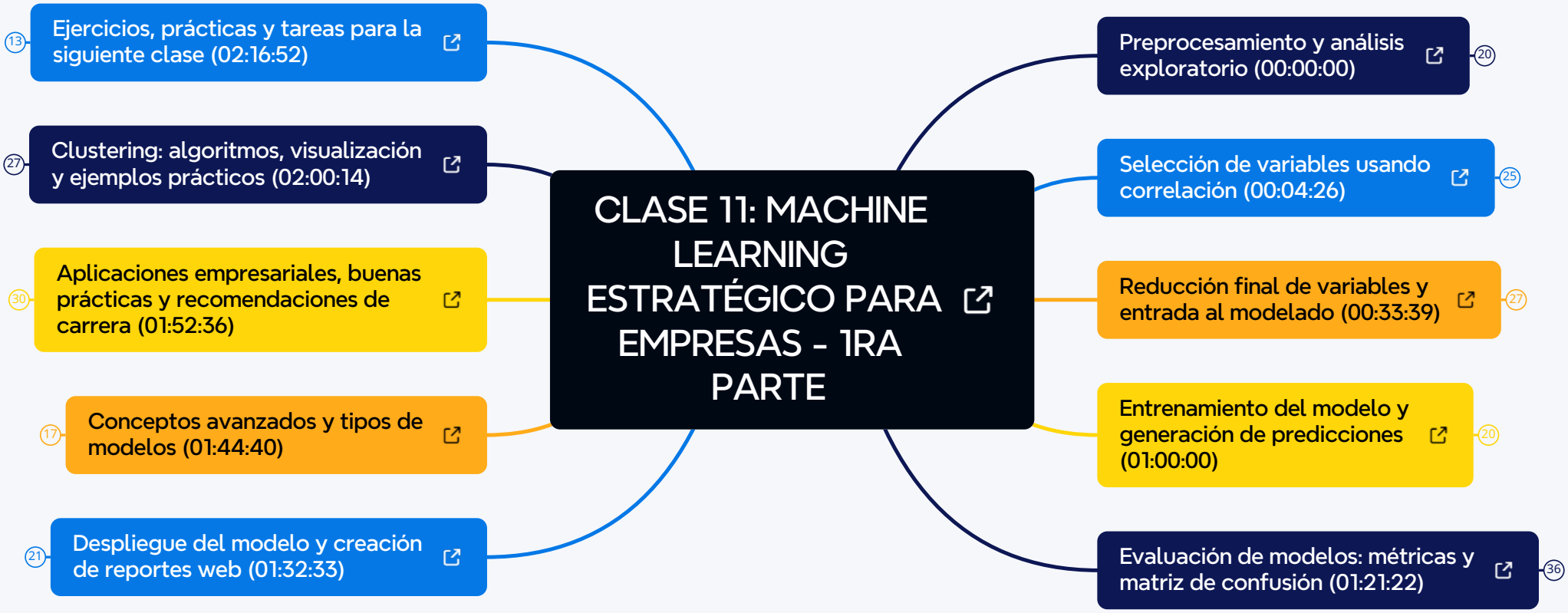
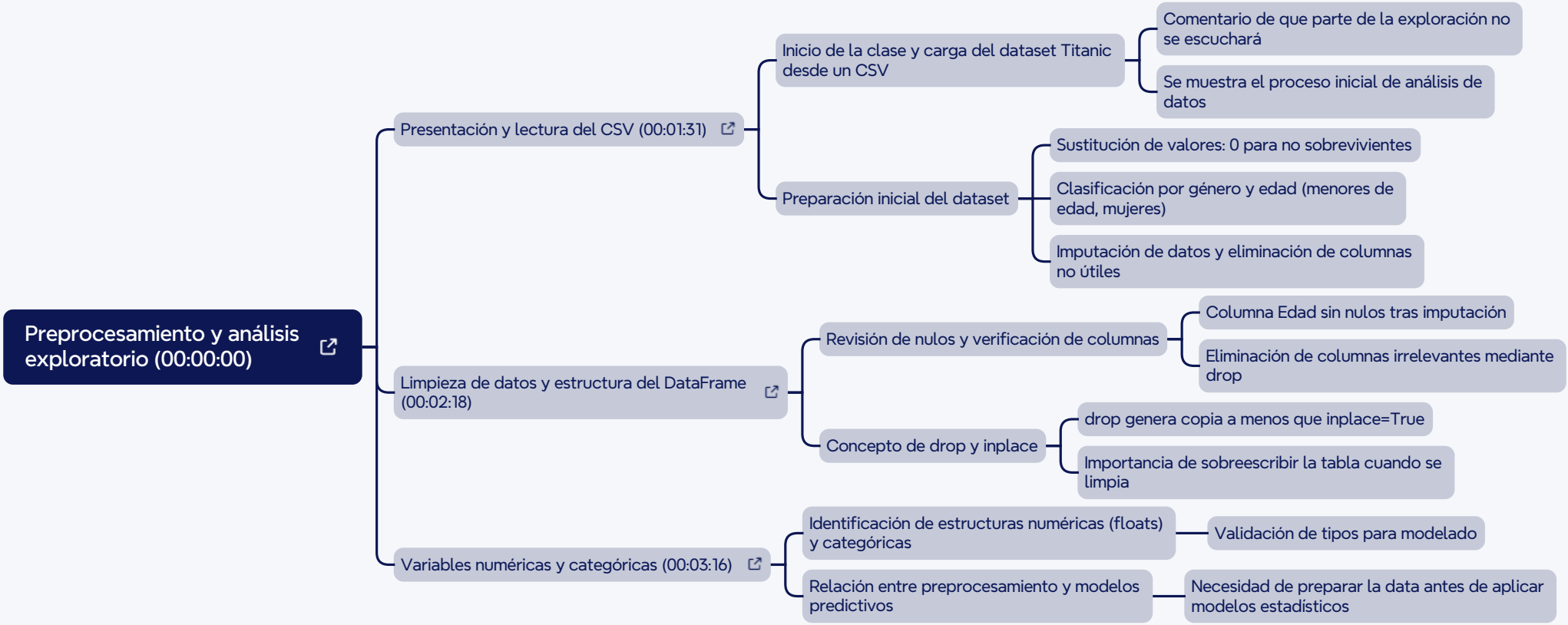
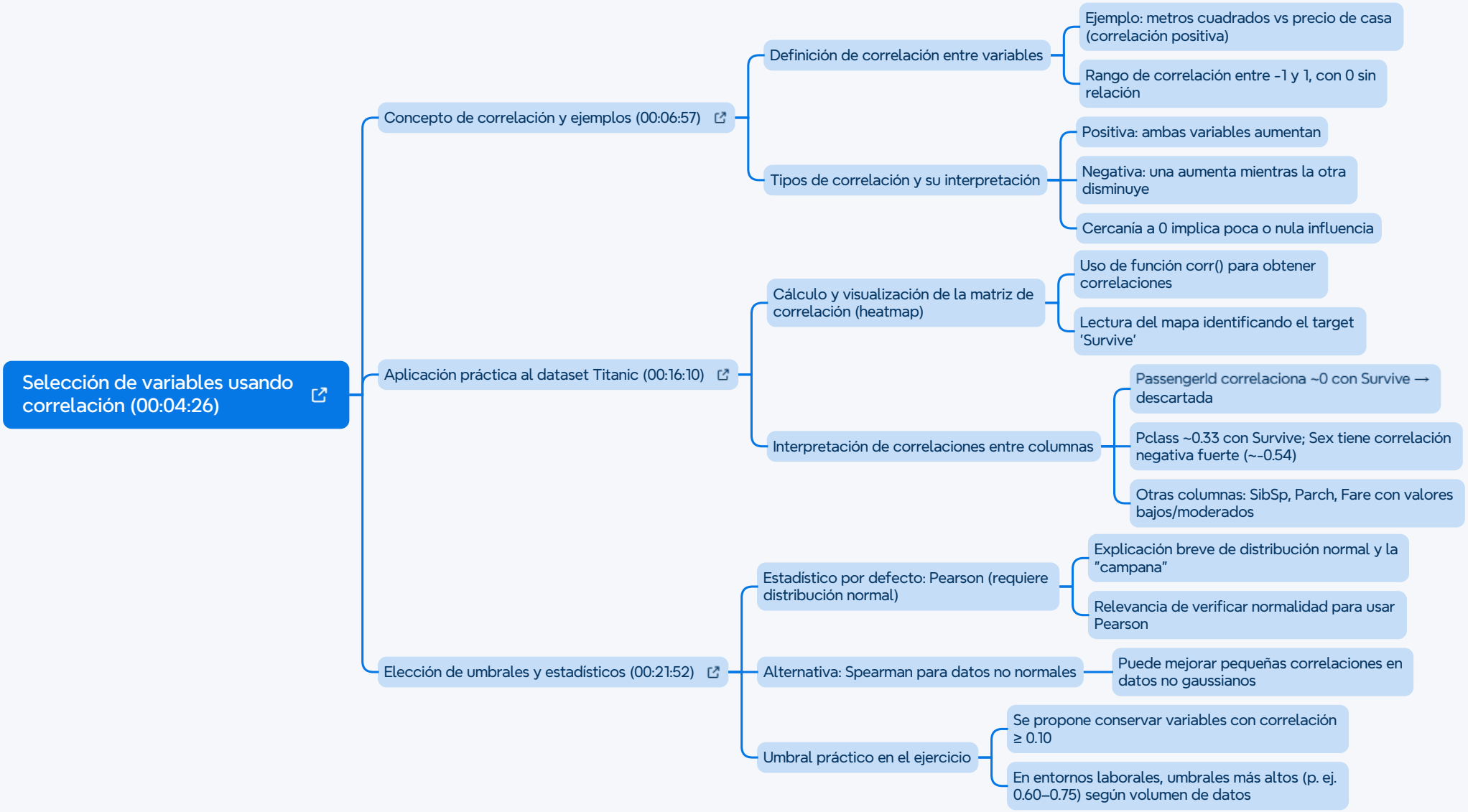
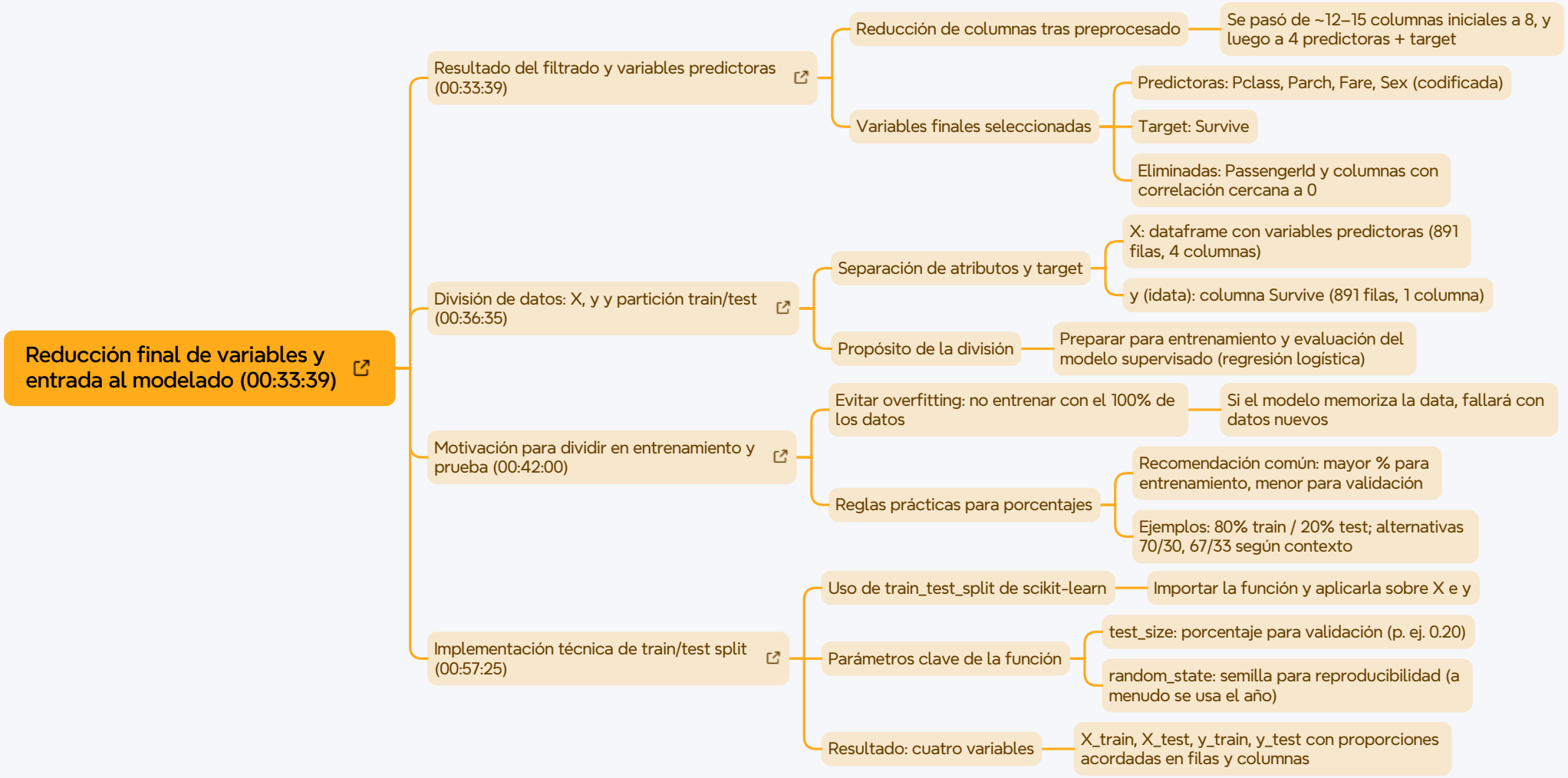
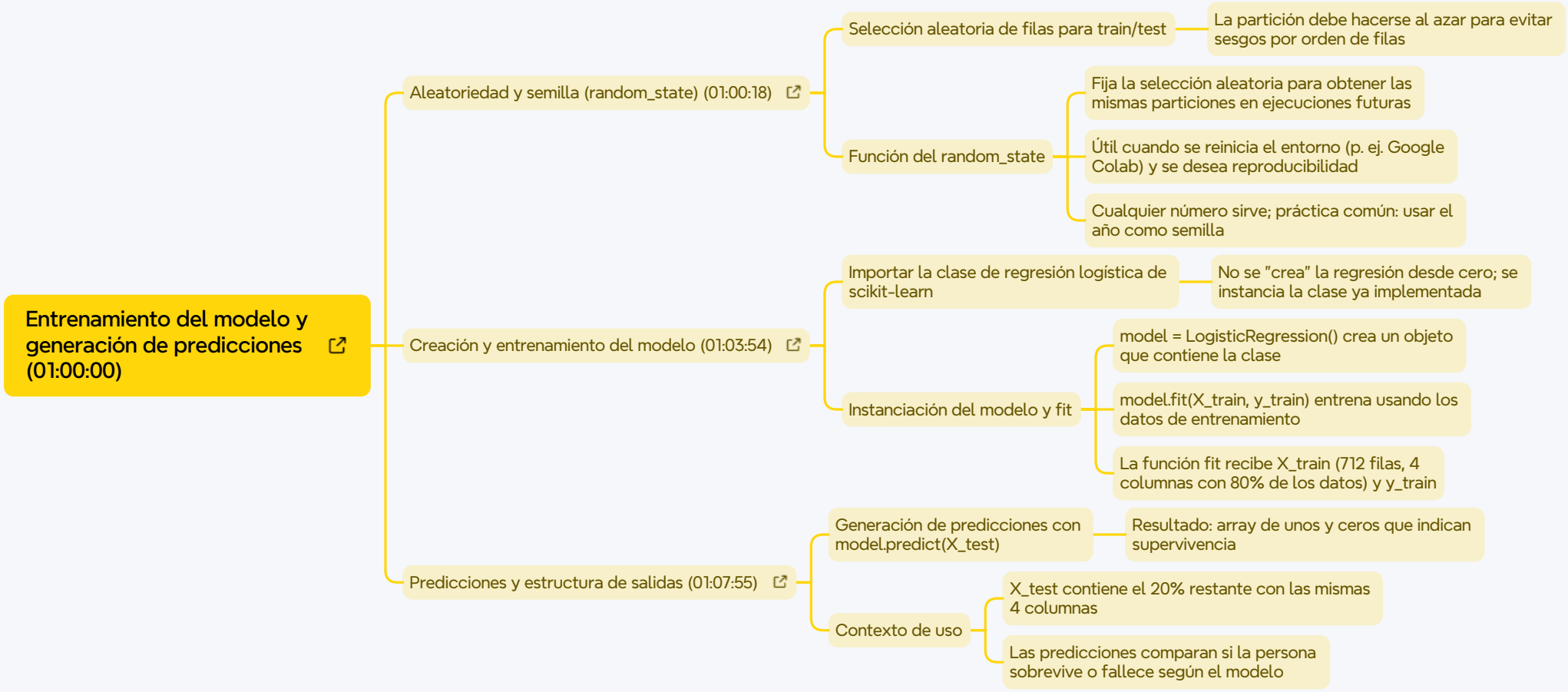
EDU MIN

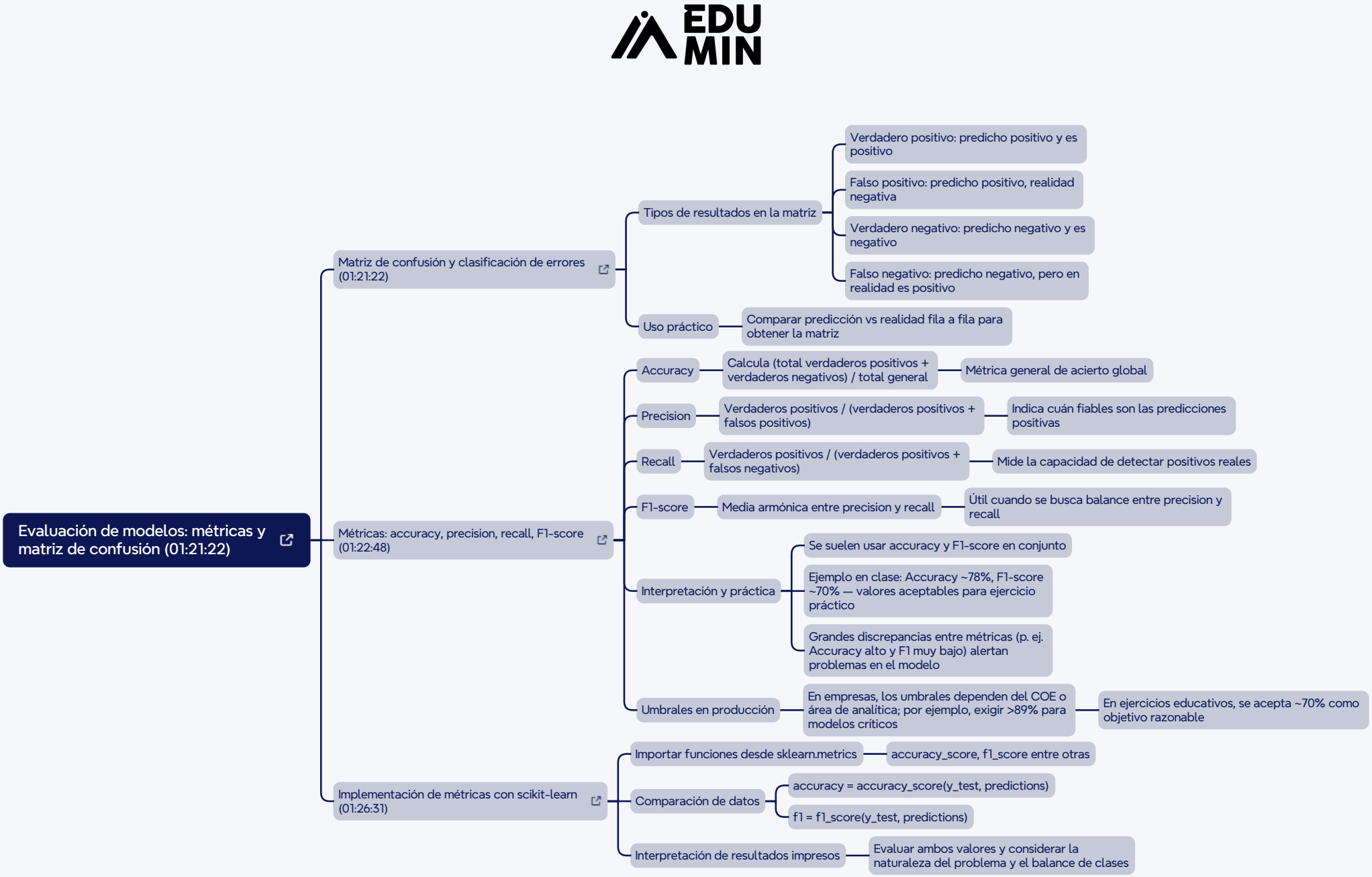

CLASE 11: MACHINE LEARNING ESTRATÉGICO PARA EMPRESAS - 1RA PARTE
Ejercicios, prácticas y tareas para la siguiente clase (02:16:52)
13
Clustering: algoritmos, visualización y ejemplos prácticos (02:00:14)
27
Aplicaciones empresariales, buenas prácticas y recomendaciones de carrera (01:52:36)
30
Conceptos avanzados y tipos de modelos (01:44:40)
17
Despliegue del modelo y creación de reportes web (01:32:33)
21
Preprocesamiento y análisis exploratorio (00:00:00)
20
Selección de variables usando correlación (00:04:26)
25
Reducción final de variables y entrada al modelado (00:33:39)
27
Entrenamiento del modelo y generación de predicciones (01:00:00)
20
Evaluación de modelos: métricas y matriz de confusión (01:21:22)
36


Preprocesamiento y análisis exploratorio (00:00:00)
Presentación y lectura del CSV (00:01:31)
Inicio de la clase y carga del dataset Titanic desde un CSV
Comentario de que parte de la exploración no se escuchará
Se muestra el proceso inicial de análisis de datos
Preparación inicial del dataset
Sustitución de valores: 0 para no sobrevivientes
Clasificación por género y edad (menores de edad, mujeres)
Imputación de datos y eliminación de columnas no útiles
Limpieza de datos y estructura del DataFrame (00:02:18)
Revisión de nulos y verificación de columnas
Columna Edad sin nulos tras imputación
Eliminación de columnas irrelevantes mediante drop
Concepto de drop y inplace
drop genera copia a menos que inplace=True
Importancia de sobreescribir la tabla cuando se limpia
Variables numéricas y categóricas (00:03:16)
Identificación de estructuras numéricas (floats) y categóricas
Validación de tipos para modelado
Relación entre preprocesamiento y modelos predictivos
Necesidad de preparar la data antes de aplicar modelos estadísticos


Selección de variables usando correlación (00:04:26)
Concepto de correlación y ejemplos (00:06:57)
Definición de correlación entre variables
Ejemplo: metros cuadrados vs precio de casa (correlación positiva)
Rango de correlación entre -1 y 1, con 0 sin relación
Tipos de correlación y su interpretación
Positiva: ambas variables aumentan
Negativa: una aumenta mientras la otra disminuye
Cercanía a 0 implica poca o nula influencia
Aplicación práctica al dataset Titanic (00:16:10)
Cálculo y visualización de la matriz de correlación (heatmap)
Uso de función corr() para obtener correlaciones
Lectura del mapa identificando el target 'Survive'
Interpretación de correlaciones entre columnas
PassengerId correlaciona ~0 con Survive → descartada
Pclass ~0.33 con Survive; Sex tiene correlación negativa fuerte (~-0.54)
Otras columnas: SibSp, Parch, Fare con valores bajos/moderados
Elección de umbrales y estadísticos (00:21:52)
Estadístico por defecto: Pearson (requiere distribución normal)
Explicación breve de distribución normal y la "campana"
Relevancia de verificar normalidad para usar Pearson
Alternativa: Spearman para datos no normales
Puede mejorar pequeñas correlaciones en datos no gaussianos
Umbral práctico en el ejercicio
Se propone conservar variables con correlación ≥ 0.10
En entornos laborales, umbrales más altos (p. ej. 0.60–0.75) según volumen de datos

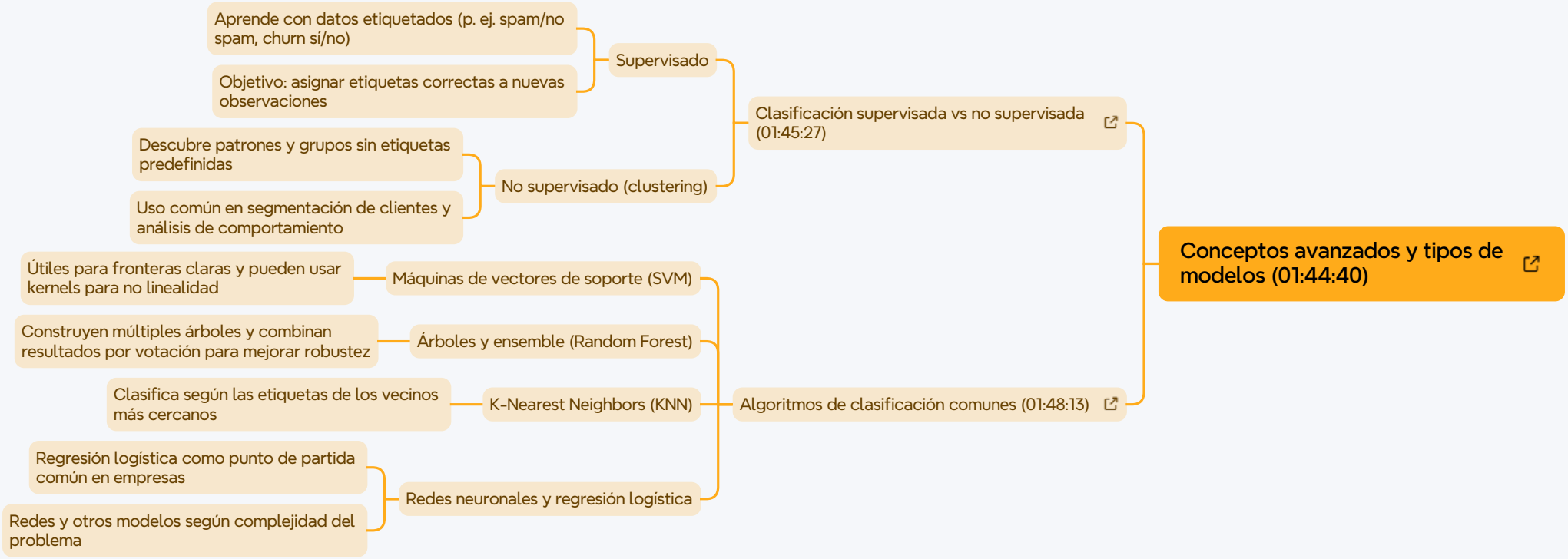

Reducción final de variables y entrada al modelado (00:33:39)
Resultado del filtrado y variables predictoras (00:33:39)
Reducción de columnas tras preprocesado
Se pasó de ~12–15 columnas iniciales a 8, y luego a 4 predictoras + target
Variables finales seleccionadas
Predictoras: Pclass, Parch, Fare, Sex (codificada)
Target: Survive
Eliminadas: PassengerId y columnas con correlación cercana a 0
División de datos: X, y y partición train/test (00:36:35)
Separación de atributos y target
X: dataframe con variables predictoras (891 filas, 4 columnas)
y (idata): columna Survive (891 filas, 1 columna)
Propósito de la división
Preparar para entrenamiento y evaluación del modelo supervisado (regresión logística)
Motivación para dividir en entrenamiento y prueba (00:42:00)
Evitar overfitting: no entrenar con el 100% de los datos
Si el modelo memoriza la data, fallará con datos nuevos
Reglas prácticas para porcentajes
Recomendación común: mayor % para entrenamiento, menor para validación
Ejemplos: 80% train / 20% test; alternativas 70/30, 67/33 según contexto
Implementación técnica de train/test split (00:57:25)
Uso de train_test_split de scikit-learn
Importar la función y aplicarla sobre X e y
Parámetros clave de la función
test_size: porcentaje para validación (p. ej. 0.20)
random_state: semilla para reproducibilidad (a menudo se usa el año)
Resultado: cuatro variables
X_train, X_test, y_train, y_test con proporciones acordadas en filas y columnas


Entrenamiento del modelo y generación de predicciones (01:00:00)
Aleatoriedad y semilla (random_state) (01:00:18)
Selección aleatoria de filas para train/test
La partición debe hacerse al azar para evitar sesgos por orden de filas
Función del random_state
Fija la selección aleatoria para obtener las mismas particiones en ejecuciones futuras
Útil cuando se reinicia el entorno (p. ej. Google Colab) y se desea reproducibilidad
Cualquier número sirve; práctica común: usar el año como semilla
Creación y entrenamiento del modelo (01:03:54)
Importar la clase de regresión logística de scikit-learn
No se "crea" la regresión desde cero; se instancia la clase ya implementada
Instanciación del modelo y fit
model = LogisticRegression() crea un objeto que contiene la clase
model.fit(X_train, y_train) entrena usando los datos de entrenamiento
La función fit recibe X_train (712 filas, 4 columnas con 80% de los datos) y y_train
Predicciones y estructura de salidas (01:07:55)
Generación de predicciones con model.predict(X_test)
Resultado: array de unos y ceros que indican supervivencia
Contexto de uso
X_test contiene el 20% restante con las mismas 4 columnas
Las predicciones comparan si la persona sobrevive o fallece según el modelo


Evaluación de modelos: métricas y matriz de confusión (01:21:22)
Matriz de confusión y clasificación de errores (01:21:22)
Tipos de resultados en la matriz
Verdadero positivo: predicho positivo y es positivo
Falso positivo: predicho positivo, realidad negativa
Verdadero negativo: predicho negativo y es negativo
Falso negativo: predicho negativo, pero en realidad es positivo
Uso práctico
Comparar predicción vs realidad fila a fila para obtener la matriz
Métricas: accuracy, precision, recall, F1-score (01:22:48)
Accuracy
Calcula (total verdaderos positivos + verdaderos negativos) / total general
Métrica general de acierto global
Precision
Verdaderos positivos / (verdaderos positivos + falsos positivos)
Indica cuán fiables son las predicciones positivas
Recall
Verdaderos positivos / (verdaderos positivos + falsos negativos)
Mide la capacidad de detectar positivos reales
F1-score
Media armónica entre precision y recall
Útil cuando se busca balance entre precision y recall
Interpretación y práctica
Se suelen usar accuracy y F1-score en conjunto
Ejemplo en clase: Accuracy ~78%, F1-score ~70% — valores aceptables para ejercicio práctico
Grandes discrepancias entre métricas (p. ej. Accuracy alto y F1 muy bajo) alertan problemas en el modelo
Umbrales en producción
En empresas, los umbrales dependen del COE o área de analítica; por ejemplo, exigir >89% para modelos críticos
En ejercicios educativos, se acepta ~70% como objetivo razonable
Implementación de métricas con scikit-learn (01:26:31)
Importar funciones desde sklearn.metrics
accuracy_score, f1_score entre otras
Comparación de datos
accuracy = accuracy_score(y_test, predictions)
f1 = f1_score(y_test, predictions)
Interpretación de resultados impresos
Evaluar ambos valores y considerar la naturaleza del problema y el balance de clases

Despliegue del modelo y creación de reportes web (01:32:33)

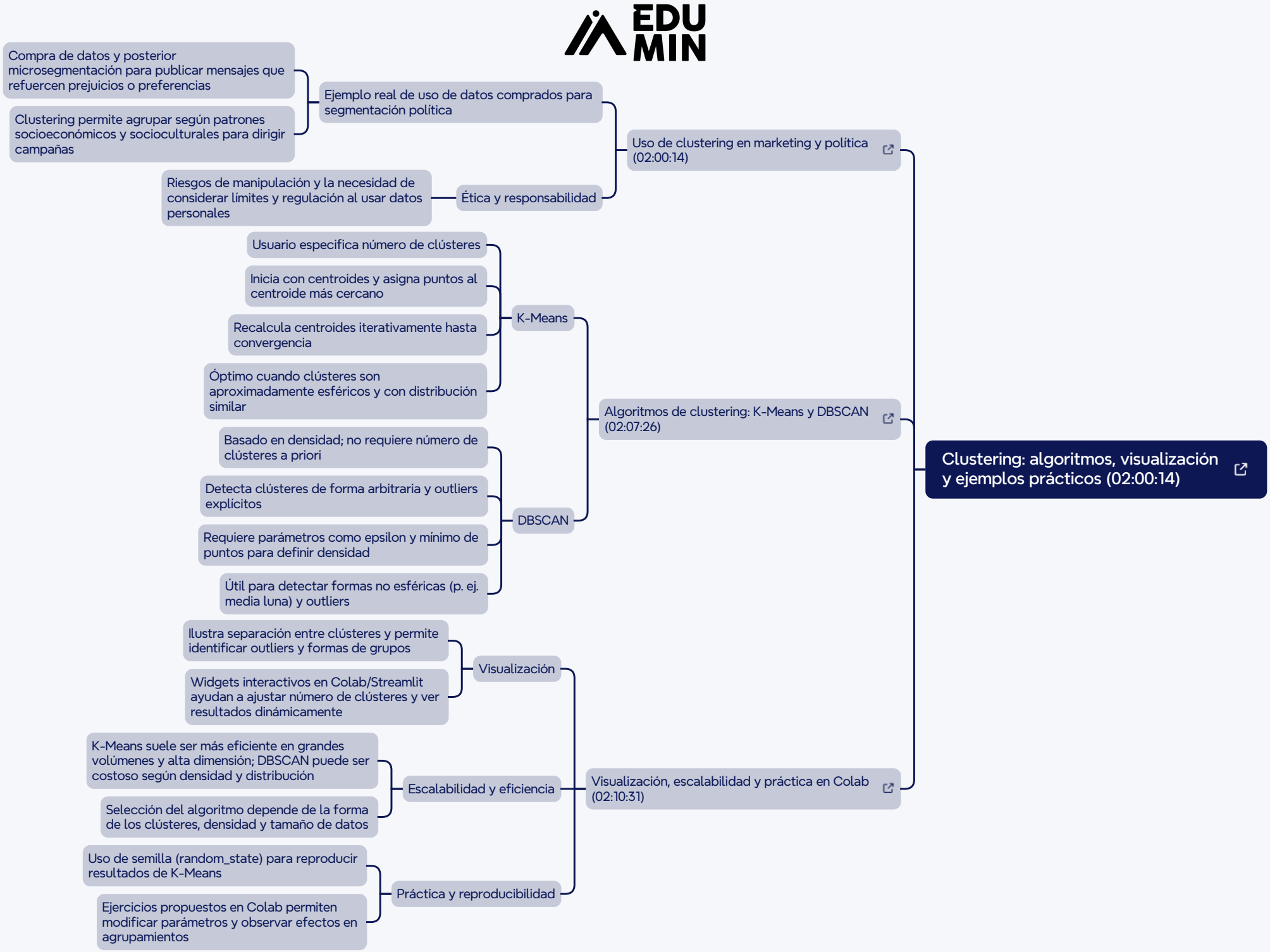
- Guardado y serialización del modelo (01:32:33)
  - Uso de joblib para exportar el modelo entrenado
    - joblib.dump(model, 'modelo.pkl') guarda el modelo en formato .pkl
    - Recomienda guardar siempre en .pkl para facilitar carga posterior
  - Ventaja
    - Permite reutilizar el modelo sin reentrenarlo y facilitar su despliegue
- Creación de una aplicación simple con Streamlit (01:36:18)
  - Streamlit como herramienta para desplegar modelos
    - Framework en Python orientado a crear aplicaciones web interactivas con pocas líneas
  - Elementos de la app para Titanic
    - Título: Predicciones Titanic
    - Widgets de entrada: selección de clase (1,2,3), número de padres/hijos, fare, género
    - Procesado de inputs y creación del DataFrame de entrada
    - Botón para ejecutar la predicción y mostrar resultado (sobrevive/no sobrevive)
  - Flujo técnico
    - Cargar modelo .pkl con joblib.load
    - Convertir inputs a la misma estructura usada en model.fit
    - Mostrar resultados e interacción visual sencilla
  - Aplicabilidad laboral
    - En muchas empresas se entrega el modelo como una app web (Streamlit u otra), o se entrega código según la preferencia del cliente
    - Roles: científicos de datos crean el modelo; ingenieros de ML despliegan en producción (contenedores, servidores, Docker)

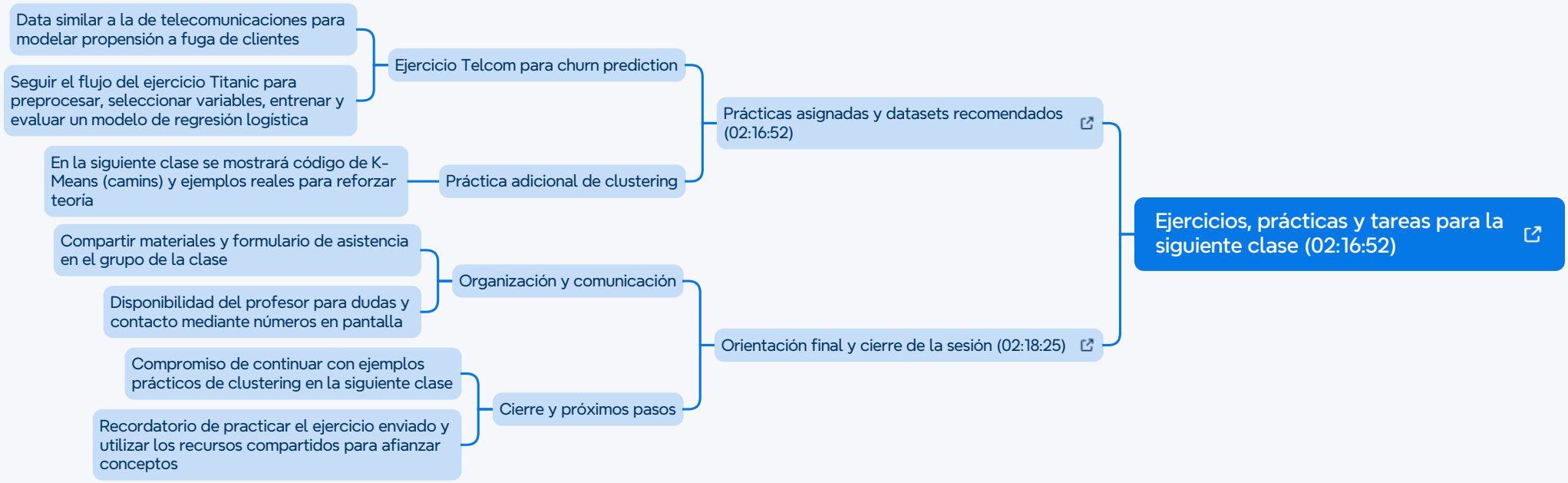

Conceptos avanzados y tipos de modelos (01:44:40)
Clasificación supervisada vs no supervisada (01:45:27)
Supervisado
Aprende con datos etiquetados (p. ej. spam/no spam, churn sí/no)
Objetivo: asignar etiquetas correctas a nuevas observaciones
No supervisado (clustering)
Descubre patrones y grupos sin etiquetas predefinidas
Uso común en segmentación de clientes y análisis de comportamiento
Algoritmos de clasificación comunes (01:48:13)
Máquinas de vectores de soporte (SVM)
Útiles para fronteras claras y pueden usar kernels para no linealidad
Árboles y ensemble (Random Forest)
Construyen múltiples árboles y combinan resultados por votación para mejorar robustez
K-Nearest Neighbors (KNN)
Clasifica según las etiquetas de los vecinos más cercanos
Redes neuronales y regresión logística
Regresión logística como punto de partida común en empresas
Redes y otros modelos según complejidad del problema



Aplicaciones empresariales, buenas prácticas y recomendaciones de carrera (01:52:36)

- Casos de uso empresarial (01:47:22)
  - Aplicaciones típicas de clasificación
    - Detección de fraude en transacciones
    - Aprobación de créditos
    - Churn prediction (abandono de clientes)
    - Clasificación de sentimiento en comentarios
  - Aplicaciones de clustering
    - Segmentación de mercado, personalización de campañas, análisis de comportamiento de usuarios
    - Ejemplo práctico en redes sociales: compra y venta de datos para segmentar audiencias políticas y dirigir mensajes específicos
- Buenas prácticas y consideraciones de datos (01:31:23)
  - Volumen y calidad de datos impactan la performance
    - Ejercicio con CSV pequeño puede dar métricas moderadas; en producción se usan datasets mucho mayores (MB a TB)
  - Elección de herramientas y plataformas
    - Cambios en herramientas (Cloudera, Databricks, Hadoop, Spark) no cambian la lógica del pipeline, pero sí la implementación y despliegue
  - Recomendaciones profesionales
    - Prepararse para distintos roles: data scientist, data engineer, ML engineer
    - Aprender despliegue y herramientas de nube facilita inserción laboral
    - Probar distintos ejemplos y practicar con notebooks y apps interactivas
  - Consejos sectoriales
    - En banca predominan modelos supervisados (regresión logística) para scoring, fraude y consumo
    - En marketing predominan técnicas no supervisadas (clustering) para segmentación y campañas
    - Trabajar en banca puede ofrecer mayor disponibilidad de datos y oportunidades de aprendizaje por rotación de personal
- Recursos, herramientas y aprendizaje continuo (01:11:46)
  - Libros y materiales recomendados
    - Recomendaciones de lecturas en Python, RPA y Big Data
  - Herramientas técnicas mencionadas
    - Google Colab para desarrollo, Visual Studio Code para entornos locales
    - joblib para serializar modelos, Streamlit para aplicaciones rápidas
  - Consejos de carrera
    - Experimentar con distintos proyectos, evitar demasiados trabajos simultáneos, especializarse según interés (modelo vs despliegue)


Clustering: algoritmos, visualización y ejemplos prácticos (02:00:14)
Uso de clustering en marketing y política (02:00:14)
Ejemplo real de uso de datos comprados para segmentación política
Compra de datos y posterior microsegmentación para publicar mensajes que refuercen prejuicios o preferencias
Clustering permite agrupar según patrones socioeconómicos y socioculturales para dirigir campañas
Ética y responsabilidad
Riesgos de manipulación y la necesidad de considerar límites y regulación al usar datos personales
Algoritmos de clustering: K-Means y DBSCAN (02:07:26)
K-Means
Usuario especifica número de clústeres
Inicia con centroides y asigna puntos al centroide más cercano
Recalcula centroides iterativamente hasta convergencia
Óptimo cuando clústeres son aproximadamente esféricos y con distribución similar
DBSCAN
Basado en densidad; no requiere número de clústeres a priori
Detecta clústeres de forma arbitraria y outliers explícitos
Requiere parámetros como epsilon y mínimo de puntos para definir densidad
Útil para detectar formas no esféricas (p. ej. media luna) y outliers
Visualización, escalabilidad y práctica en Colab (02:10:31)
Visualización
Ilustra separación entre clústeres y permite identificar outliers y formas de grupos
Widgets interactivos en Colab/Streamlit ayudan a ajustar número de clústeres y ver resultados dinámicamente
Escalabilidad y eficiencia
K-Means suele ser más eficiente en grandes volúmenes y alta dimensión; DBSCAN puede ser costoso según densidad y distribución
Selección del algoritmo depende de la forma de los clústeres, densidad y tamaño de datos
Práctica y reproducibilidad
Uso de semilla (random_state) para reproducir resultados de K-Means
Ejercicios propuestos en Colab permiten modificar parámetros y observar efectos en agrupamientos


Ejercicios, prácticas y tareas para la siguiente clase (02:16:52)
Prácticas asignadas y datasets recomendados (02:16:52)
Ejercicio Telcom para churn prediction
Data similar a la de telecomunicaciones para modelar propensión a fuga de clientes
Seguir el flujo del ejercicio Titanic para preprocesar, seleccionar variables, entrenar y evaluar un modelo de regresión logística
Práctica adicional de clustering
En la siguiente clase se mostrará código de K-Means (camins) y ejemplos reales para reforzar teoría
Orientación final y cierre de la sesión (02:18:25)
Organización y comunicación
Compartir materiales y formulario de asistencia en el grupo de la clase
Disponibilidad del profesor para dudas y contacto mediante números en pantalla
Cierre y próximos pasos
Compromiso de continuar con ejemplos prácticos de clustering en la siguiente clase
Recordatorio de practicar el ejercicio enviado y utilizar los recursos compartidos para afianzar conceptos